

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Лабораторная работа 3

По дисциплине Объектно-ориентированное программирование

Тема работы Алгоритмы с бинарными деревьями

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Иванов С.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 РЕЗУЛЬТАТ РАБОТЫ.....	4
1.1 Алгоритмы обхода дерева	4
1.2 Дерево – математическое выражение	9
2 КОНТРОЛЬНЫЕ ВОПРОСЫ	11
ЗАКЛЮЧЕНИЕ	12

ВВЕДЕНИЕ

В отчёте представлено решение лабораторной работы. В тексте работы представлены результаты реализации алгоритмов и ответы на контрольные вопросы.

Цель — изучить алгоритмы работы с деревьями. Реализовать средствами ООП дерево и рекурсивные функции, выполняющие обход дерева в прямом, обратном, концевом порядке. Вычислить значение выражения, заданного деревом.

Задание на лабораторную работу

1. Реализовать средствами ООП дерево и рекурсивные функции, выполняющие обход дерева в прямом, обратном, концевом порядке.

Выполнить расчет примера.

Обход дерева в прямом порядке *abdecf*.

Обход дерева в обратном порядке *dbeacf*.

Обход дерева в концевом порядке *debfga*.

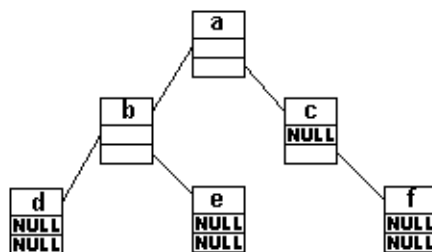


Рисунок 1 — Пример дерева

2. Вычислить значение выражения, заданного деревом. Вычисление выполнять в порядке концевого обхода.

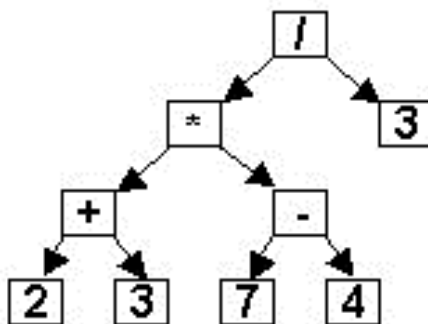


Рисунок 2 — Пример выражения в дереве

1 РЕЗУЛЬТАТ РАБОТЫ

1.1 Алгоритмы обхода дерева

Для алгоритмах на деревьях сперва был создан класс дерева. В статичной (общей) части храниться последний рассчитанный порядок обхода `list` и корневой элемент `Node`. У одного элемента есть значение, левый и правый элементы (рисунок 3).

```
1 public class Tree
2 {
3     public Tree Left;
4     public Tree Right;
5     public char Value;
6     public bool flag = false;
7     public static List<char> list = new List<char>();
8     public static Tree Node;
9
10    public Tree()
11    {
12        Node = null;
13    }
14
15    public Tree(Tree left, Tree right, char value)
16    {
17        Left = left;
18        Right = right;
19        Value = value;
20    }
```

Рисунок 3 — Поля и конструктор дерева

Далее были определены и реализованы методы добавления элемента в дерево `Add` (рисунок 4). Вставляем элемент по приоритету: определяем корневой элемент, если уже определён определяем левый элемент, если и тот уже определён, заполняем правый элемент, если и правый тоже определён, добавляем к правому рекурсивно.

```
1  public void Add(ref Tree node, char value)
2  {
3      if (node == null) {
4          node = new Tree(null, null, value);
5          flag = false;
6      }
7      else if (node.Value != '.' && !flag)
8      {
9          if (node.Left != null)
10         {
11             Add(ref node.Left, value);
12             if (!flag)
13             {
14                 if (node.Right != null)
15                 {
16                     Add(ref node.Right, value);
17                 }
18                 else
19                 {
20                     node.Right = new Tree(null, null, value);
21                     flag = true;
22                 }
23             }
24         }
25         else
26         {
27             node.Left = new Tree(null, null, value);
28             flag = true;
29         }
30     }
31 }
```

Рисунок 4 — Добавление элемента в дерево

На рисунке 5 вспомогательный метод **Fill** заполнения дерева данными из массива символов.

```
1 public void Fill(ref Tree node, List<char> chars)
2 {
3     for (int i = 0; i < chars.Count; i++)
4     {
5         flag = false;
6         Add(ref node, chars[i]);
7     }
8 }
```

Рисунок 5 — Заполнение дерева

На рисунке 6 реализация рекурсивного отображения в консоль структуры дерева.

```
1 public void PrintTree(Tree node)
2 {
3     if (node == null) return;
4     if (node.Value != '.')
5     {
6         Console.WriteLine("value: {0}, left: {1}, right: {2}", node.Value,
7             ↪ node.Left.Value, node.Right.Value);
8     }
9     PrintTree(node.Left);
10    PrintTree(node.Right);
11 }
```

Рисунок 6 — Вывод дерева в консоль

Последним в реализации дерева стало написание трёх способов обхода дерева (рисунок 7). Прямой ход: начинаем с самого элемента, затем левая и правая ветви. Обратный ход: левая ветвь, непосредственно сам элемент и правая ветвь. Концевой ход: лево, право, элемент.

```

1  public static List<char> TreeWalkDirect(Tree node)
2  {
3      /* NLR */
4      if (node.Value == '.') return new List<char>();
5
6      list.Add(node.Value);
7      TreeWalkDirect(node.Left);
8      TreeWalkDirect(node.Right);
9
10     return list;
11 }
12
13 public static List<char> TreeWalkFinal(Tree node)
14 {
15     /* LRN */
16     if (node.Value == '.') return new List<char>();
17
18     TreeWalkFinal(node.Left);
19     TreeWalkFinal(node.Right);
20     list.Add(node.Value);
21
22     return list;
23 }
24
25 public static List<char> TreeWalkReverse(Tree node)
26 {
27     /* LNR */
28     if (node.Value == '.') return new List<char>();
29     var result = TreeWalkReverse(node.Left);
30     result.Add(node.Value);
31     result.AddRange(TreeWalkReverse(node.Right));
32     list = result;
33     return result;
34 }

```

Рисунок 7 — Три способа обхода дерева

Для тестов был написан код (рисунок 8), который заполняет дерево, отображает его структуру и выполняет три варианта его обхода.

```
1 static void Main(string[] args)
2 {
3     Tree t = new Tree();
4     t.Fill(ref Tree.Node, new List<char>() { 'a', 'b', 'd', '.', '.', 'e',
5     ↪   '.', '.', 'c', '.', 'f', '.', '.' });
6     t.PrintTree(Tree.Node);
7
8     Console.WriteLine("Direct order");
9     Tree.TreeWalkDirect(Tree.Node);
10    Console.WriteLine(string.Join(" ", Tree.list));
11    Tree.list.Clear();
12
13    Console.WriteLine("Reverse order");
14    Tree.TreeWalkReverse(Tree.Node);
15    Console.WriteLine(string.Join(" ", Tree.list));
16    Tree.list.Clear();
17
18    Console.WriteLine("The final order");
19    Tree.TreeWalkFinal(Tree.Node);
20    Console.WriteLine(string.Join(" ", Tree.list));
21    Tree.list.Clear();
22 }
```

Рисунок 8 — Код запуска алгоритмов

На рисунке 9 представлен результат работы программы.



```
Microsoft Visual Studio Debug Console
value: a, left: b, right: c
value: b, left: d, right: e
value: d, left: ., right: .
value: e, left: ., right: .
value: c, left: ., right: f
value: f, left: ., right: .
Direct order
a b d e c f
Reverse order
d b e a c f
The final order
d e b f c a
D:\ITMO.PRIVATE\4_OOP\Labs\Lab03\Tree\bin\Debug\Tree.exe (process 40920) exited with code 0 (0x0).
```

Рисунок 9 — Результат работы программы

1.2 Дерево – математическое выражение

В классе `Tree` был добавлен метод для вычисления дерева как математического выражения (рисунок 10). Если узел – это число, иначе считаем левый и правый узлы рекурсивно и возвращаем результат оператора.

```
1  public static int CalcTree(Tree node)
2  {
3      if (char.IsDigit(node.Value)) return node.Value;
4      int a = CalcTree(node.Left);
5      int b = CalcTree(node.Right);
6      switch (node.Value)
7      {
8          case '+': return a + b;
9          case '-': return a - b;
10         case '*': return a * b;
11         case '/': return a / b;
12         default: return 0;
13     }
14 }
```

Рисунок 10 — Вычисление дерева

Для тестов был написан код (рисунок 11), который заполняет дерево, отображает его структуру и выполняет вычисления.

```

1  static void Main(string[] args)
2  {
3      Tree t = new Tree();
4      t.Fill(ref Tree.Node, new List<char>() {
5          '/', '*', '+', '2', '.', '.', '3', '.', '.', '-', '7',
6          '.', '.', '4', '.', '.', '3', '.', '.'
7      });
8      t.PrintTree(Tree.Node);
9
10     Console.WriteLine("The final order");
11     Tree.TreeWalkFinal(Tree.Node);
12     Console.WriteLine(string.Join(" ", Tree.list));
13     Tree.list.Clear();
14
15     Console.WriteLine("The result of the expression is");
16     Console.WriteLine(Tree.CalcTree(Tree.Node));
17 }

```

Рисунок 11 — Код запуска алгоритмов

На рисунке 12 представлен результат работы программы.

```

Microsoft Visual Studio Debug Console
value: /, left: *, right: 3
value: *, left: +, right: -
value: +, left: 2, right: 3
value: 2, left: ., right: .
value: 3, left: ., right: .
value: -, left: 7, right: 4
value: 7, left: ., right: .
value: 4, left: ., right: .
value: 3, left: ., right: .
The final order
2 3 + 7 4 - * 3 /
The result of the expression is
5
D:\ITMO.PRIVATE\4_OOP\Labs\Lab03\Tree\bin\Debug\Tree.exe (process 31156) exited with code 0 (0x0).

```

Рисунок 12 — Результат работы программы

2 КОНТРОЛЬНЫЕ ВОПРОСЫ

1) Какие ограничения и условия на применения алгоритма обхода?

Для корректной работы алгоритмов обхода дерево должно быть бинарным, каждый узел имеет ≤ 2 потомков. Необходимо корректное определение структуры узла (значение, левый и правый потомки).

2) К какому классу сложности относится алгоритм обхода?

Все три обхода (прямой, обратный, концевой) имеют линейную временную сложность $O(n)$, где n — количество узлов, так как каждый узел посещается ровно один раз.

3) Какие конструкции ООП C# применяются для реализации прямого, обратного и концевого обхода?

Для реализации используются класс `Tree` с полями `Left`, `Right`, `Value` (инкапсуляция). Статические поля (`list`, `Node`) хранят результаты и корень. Рекурсивные методы `TreeWalkDirect`, `TreeWalkReverse`, `TreeWalkFinal` реализуют алгоритмы обхода.

4) Как оценивается сложность алгоритма обхода?

Временная сложность алгоритмов обхода составляет $O(n)$. Пространственная сложность в худшем случае также $O(n)$ из-за глубины рекурсии для вырожденного дерева и размера списка результатов.

5) Какие задачи решаются с помощью алгоритма обхода?

Прямой обход (NLR) используется для копирования дерева, создания префиксной записи выражения. Обратный обход (LNR) позволяет получить отсортированную последовательность для бинарного дерева поиска, инфиксную запись. Концевой обход (LRN) применяется для вычисления выражения (постфиксная запись), удаления дерева, подсчёта высоты.

6) Как можно оценить точность алгоритма обхода?

Точность определяется соответствием порядка обхода определению (NLR, LNR, LRN). Проверяется сравнением с эталонным порядком для небольшого дерева или с помощью модульных тестов.

7) Какой обход дерева применяется для вычисления выражения?

Для вычисления выражения применяется концевой обход (LRN), так как он позволяет вычислить значения левого и правого поддеревьев до применения оператора (постфиксная запись).

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы изучены и реализованы такие алгоритмы работы с деревьями, как прямой, обратный и концевой обход, а также алгоритм расчета выражения, заданного деревом.